

CS1010E: Programming Methodology

Assignment 3: Recursions, Search, Sort and MDA

Instructions

Submission Instructions

Read **ALL** instructions carefully.

1. Submission Content:

- Each question will be given a template file (e.g., `aN_qM_template.py`).
 - You should work on your file locally before submitting to Coursemology.
- Comment out all the given **test cases** and **import** statements before copying the content of the entire file to Coursemology, *replacing* any content on Coursemology:
 - On template file: **CTRL + A** followed by **CTRL + C** (i.e., *copy all*).
 - On Coursemology: **CTRL + A** followed by **CTRL + V** (i.e., *replace all*).
- If any implementation is given, you are not allowed to change them.
- You are not allowed to *add, remove, or modify* any parameters from functions.
- You are not allowed to change the *name* of the given function.
- Unless otherwise stated, you are not allowed to add/change **import**.
- The deadline for this assignment is Friday 31/3/2023 at 23:59.

2. Finalizing Submissions:

- You are responsible for finalizing your submission.
- Late submission sadly **is not** allowed.
- You should not submit at the last minute since Coursemology is not designed to handle large workload.
- If you see that Coursemology is *under maintenance*, it is due to large workload and not an actual maintenance. No extension will be given.

3. Testing:

- Do not call the required functions when you copy the code to Coursemology as it will be called automatically during testing.
- Your function should **return** the result instead of **print**.
- Test cases are called in succession (i.e., *no reverting of global state*).
- Passing all test cases (*public or private*) does not guarantee full marks as we will do manual evaluation on other aspects such as code quality.
- Questions 4 would be evaluated manually.
- You may assume that the code will only be tested on valid inputs that follow assumption. Do NOT put any checks for inputs with respect to the assumptions.

4. Good Programming Practice:

- Codes that are redundant or too long may be penalized.
 - Restrictions must not be violated.
 - Each test case (*public or private*) should complete within a reasonable amount of time typically less than 30 seconds for Q1,Q2 and less than 2 seconds for Q3,Q5.
-

Failure to follow any of the instructions above will result in deduction of your marks.

Files

- Assignment03.pdf
- a3_q1_template.py
- a3_q2_template.py
- a3_q3_template.py
- a3_q4_template.py
- a3_q5_template.py

Coursemology

- Assignment 3: Iterations, Recursions, Search, Sort, MDA and Random

- **This assignment contains 5 questions. Questions 1 and 5 are obligatory and each conveys 1 CA mark.**
- **The deadline for this assignment is Friday 31/3/2023 at 23:59.**
- **Delayed submission is not allowed for this assignment.**

Question 1: Gibonacci

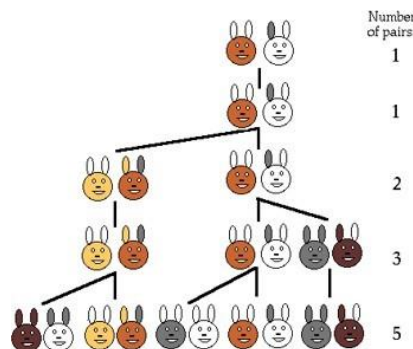
[1 CA Mark]

You may have learned in Lecture 04 that Fibonacci sequence is an answer to the question of

How many adult rabbit pairs are there at month $m \geq 0$ when:

- (1) The population begins in the first month with a pair of newborn rabbits.
- (2) Rabbits reach reproductive age after one month
- (3) In any given month, every rabbit of reproductive age mates with another rabbit of reproductive age.
- (4) Exactly one month after two rabbits mate, they produce one male and one female rabbit.
- (5) Rabbits never die or stop reproducing.

We can answer the question by the recursive formula: $f_0 = 0, f_1 = 1$ and $f_n = f_{n-1} + f_{n-2}$.



Here we have a *slightly* different problem. We make modification to the recursive formula as follows: $g_0 = 0, g_1 = 0, g_2 = 1$ and $g_n = g_{n-1} + g_{n-3}$. Note that instead of f_{n-1} and f_{n-2} , we have g_{n-1} and g_{n-3} . Why do we use this formula? Let's just say that maybe the rabbit becomes adult after 2 months instead of 1 month. We call this sequence the Gibonacci sequence. This is explained in the image below:

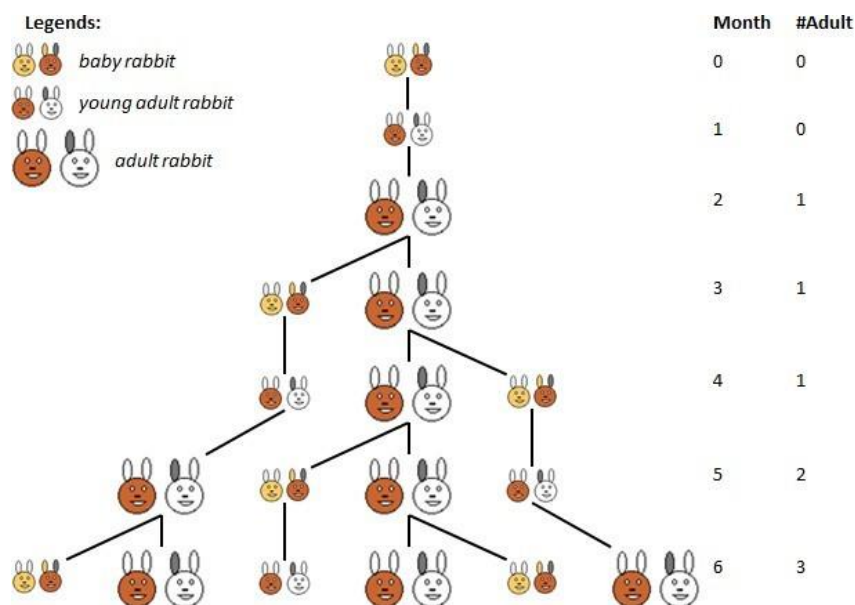


Figure 2: The Gibonacci sequence explained.

1.1 Recursion

Question

Write a *recursive* function `gibonacci_R(n)` to compute g_n by using recursion without using any repetition statement.

Restrictions

- You may not use iterative constructs (*e.g.*, loop, list comprehensions, *etc.*) to solve this.
- The function `gibonacci_R` must be *recursive* (*i.e.*, it calls itself). The use of any recursive helper functions will not be counted as being recursive.

Assumptions

- $n \geq 0$

Sample Run #1

```
1 >>> print(gibonacci_R(4))
2 1
```

Sample Run #2

```
1 >>> print(gibonacci_R(10))
2 13
```

Sample Run #3

```
1 >>> print(gibonacci_R(15))
2 88
```

1.2 Iteration

Question

Write an *iterative* function `gibonacci_I(n)` to compute g_n without using any recursion.

Restrictions

- 1.2.1 You may not use recursive function(s) to solve this.

Assumptions

- 1.2.2 $n \geq 0$

Hint

- 1.2.3 We showed the iterative Fibonacci in Lecture 06, can you modify it to Gibonacci?

Sample Run #1

```
1 >>> print(gibonacci_I(4))  
2 1
```

Sample Run #2

```
1 >>> print(gibonacci_I(10))  
2 13
```

Sample Run #3

```
1 >>> print(gibonacci_I(15))  
2 88
```

1.3 Maximum

Reflections

Find the largest output number that your functions can compute within 1 minute in Question 1.1 and Question 1.2. You don't have to find an exact tight bound, but something close is good enough. Which version is better and in what ways? Write your thoughts in the comment part of your .py file.

Notes

- Use scientific notation (*e.g.*, 2×10^{31}) if your answer is too big.
- If it is still too big, you may reduce the time (*e.g.*, 30 seconds instead of 1 minutes) and mention that in your reflections.

Question 2: Monte Carlo

Monte Carlo simulations are used to model the probability of different outcomes in a process that cannot be easily predicted due to the presence of random variables. There are many uses for Monte Carlo and they are highly beneficial when the precise answer cannot be found easily or when computation of the precise answer is too computationally expensive when approximate answers are precise enough.

2.1 Approximating π

The constant π is used in mathematics, physics, and other related fields such as engineering extensively. In this exercise, you are going to compute an approximation of the constant π .

Gambler Jack is no mathematician. His friends laugh at him and make a bet that Jack does not know the number π , not even the first three most significant digits. Jack is going to lose the bet but his girlfriend, Jane, is an accountant and she is going to help. She suggested Jack the following idea.

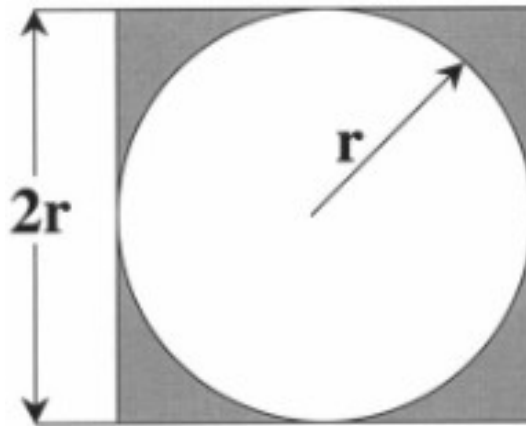


Figure 3: A circle with radius r inside a square with sides of length $2r$.

1. Use a circular dartboard of radius r inside a square of length $2r$ as shown in Figure 3.
2. Throw n number of darts *randomly* onto the dartboard.
3. Count the number m of darts that falls inside the circle.
4. The ratio of m over n is approximately a quarter of a circle.

If you are still lost, the conversation above can be distilled into the following equation for computing the probability p of the dart landing inside the circle:

$$p = \frac{\text{area of circle}}{\text{area of square}} = \frac{\pi r^2}{(2r)^2} = \frac{\pi}{4}$$

You can approximate the probability by using the following formula:

$$p \approx \frac{\text{number of darts inside circle}}{\text{total number of darts thrown}} \approx \frac{m}{n}$$

Since this is an approximation, your answer may not be close to π . For instance, if I throw 10 darts and 8 of them lands inside the circle then $\pi \approx (8/10) \times 4 = 3.2$.

Question

Write the function `monte_carlo_pi(n)` which returns an approximation of π by throwing the darts n times. Theoretically, the more darts you throw, the more accurate your π is.

Note

- Remember that you are not allowed to add/change `import` but you are already given the `random` library using `from random import *`.

Sample Run #1

```
1 >>> print(monte_carlo_pi(10))
2 3.6
3 >>> print(monte_carlo_pi(100))
4 3.32
5 >>> print(monte_carlo_pi(1000))
6 3.084
7 >>> print(monte_carlo_pi(10000))
8 3.1348
9 >>> print(monte_carlo_pi(100000))
10 3.1412
11 >>> print(monte_carlo_pi(1000000))
12 3.1410808
13 >>> print(monte_carlo_pi(10000000))
14 3.1413096
```

Note that your output may be different from the value shown above due to the use of random number generator.

Steps

The algorithm used to approximate the value of π using the method described earlier is as follows:

- Let the radius of the circle be r and the length of the square be $2r$ for some chosen value of r .
- Generate two random numbers x and y such that the point is inside the square (*i.e.*, $-r \leq x, y \leq r$). This represents the position of our darts as the coordinate (x, y) . You may use the function in the package `random` that starts with the letter `u`.
- Calculate the distance d from the dart position to the centre of the circle.
- If $d \leq r$, then the point (x, y) is inside the circle. Otherwise, the point (x, y) falls outside the circle.
- Repeat Step 2 for n times, keeping track of the number of points inside circle k . Once you have repeated Step 2 for n times, continue with Step 6.
- Compute $p = k/n$.
- For large enough n , the value of π can be approximated by $4p$.

Question 3: Sort a String (S&S)

A string is a list of characters indeed.

```
>>> b='abcdefgh'
>>> b[3]
'd'
>>> b[5]
'f'
>>>
```

You are going to develop a function to **sort** a string, but **without** repeating the possible **duplicate** characters. i.e., if there is more than one instance of a character, only one should appear in your sorted output result. e.g.:

```
>>> Z= str_sorter('coffee committee')
>>> print(Z)
' cefimot'
```

It is allowed to convert your string to list, process it, and convert it back to string, using the casting functions (e.g., str, and list), however, your output result **must** be a pure **string**, not a list. Mind the spaces too. Space is a respectable character.

Examples:

```
>>> str_sorter('zzztttt ')
'tz'

>>>
str_sorter('ab#cs4433321**zzxx')
'##*1234abcsxz'

>>> str_sorter('hello world')
'dehlorw'

>>> str_sorter('HELLO hello')
'EHL0ehlo'
```


Question 4: Sort and Solve the 7-Puzzle

Here you have a 7-Puzzle, /// means the empty cell. We can move any vertical or horizontal neighbor tile to the empty cell. (below in the example, we can call it the start state)

1	6	2	4
5	3	///	7

Lets have some fun and sort it like this: (we can call it the goal or object state)

1	2	3	4
5	6	7	///

This is a physical bigger one:



Wait a minute. We would like to have a computer program to solve any 7-puzzle for us. How? The main data structure would be a 2x4 matrix. Instead of /// we put a '0' there. So, this matrix will be numerical.

The main function is something like this:

```
def sort_7_puzzle( mat1 ):
    ret sw1
```

- input: mat1, a 2x4 integer matrix that we're going to sort.
- output: sw1, a Boolean switch, if the 7-puzzle is solved, it returns 'true', otherwise it returns 'false'
- **Approach/ Algorithm:**
 1. Loop_counter =1
 2. Loop_end_limit= 200 (for example, that's up to you to set, mine is 4000!)
 3. check if the 7-puzzle is already solved. If yes, return 'true' and stop.
 4. randomly select one of the vertical or horizontal neighbours of the empty cell (0), and swap them
 5. add one to the loop_counter
 6. if the loop_counter has passed the loop end limit, return 'false' and stop.
 7. Go to step 3 to do the loop once again

You need a function to check if the 7-puzzle is solved or not, for example:

```
def is_7_solved( mat1 ):
    # ----check if the mat1 is a sort and solved matrix or not
    # you simply can subtract your input matrix from the sorted 7-puzzle:
    # [ [1,2,3,4] , [5,6,7,0] ], if the result is an all-zero matrix (e.g., summation of all elements is 0),
    # so this is a solved 7-puzzle
    return true
    # otherwise, input matrix doesn't show a sorted/solved 7-puzzle, so
    return false
```

Note: element-wise comparison is fine too.

in your main module, you should:

- Declare the 7-puzzle matrix: mat2
- Print it using pprint
- Call sw2 = sort_7_puzzle(mat2)
- Sw2 is either true or false. Print an appropriate message along with the final mat2 matrix, again use pprint.

You may ask why moving randomly? Is it fair?

- Well, sometimes being random is not that bad!
- It's easy to implement
- In fact, randomness is a part of many complex programs.

Now, you may ask how to implement the next random move?

We have got two respective cases here: when the empty cell is in the corners, vs. when it is in the middle.

1. Empty cell in the corner: you have got 2 possible moves, horizontal and vertical.

Y	X	Z	T
U	W	V	///

2. Empty cell in the middle: you have got 3 possible moves, left, right and vertical.

A	B	C	D
E	F	///	G

How to generate random numbers?

Any programming language has got some functions and libraries to generate (pseudo) random numbers. Why pseudo? Look it up, please. You can have different random numbers (type, distribution, etc) using those functions. The function/method randrange(n) from library/ package/ class random, returns a random integer number between 0 and n-1.

```
import random
a= random.randrange( n )
```

hints: (if I were you, I would ...)

- 1- Empty cell in the corner: set n=2, call random.randrange(n), always assume 0= horizontal, 1= vertical move.
- 2- Empty cell in the middle: set n=3, call random.randrange(n), always assume 0= to left, 1= vertical, 2= to right move.

Then employing a couple of if/elif/else instructions, you can make it. So, you may define a function to do the next move for you:

```
def next_move(mat):  
    return mat
```

This is where you use random numbers to decide about the next move. Think when your '0' is in the corner and when it is in the middle.

Another approach (I used that, it's easier to implement) to write that function up is ignoring the differences between the empty cell in the corners and in the middle states. Using this approach, you generate a random number between 0 and 2 and try to do the swap. When the random number is '1' and you swap the empty cell vertically, it always works. But in the cases of swapping with right or left, the move might be impossible. In such cases, you just ignore the swap and the matrix returned would be similar to the matrix entered, and you just try your chance in the next iteration. Don't worry, machines are fast. For instance:

- In `next_move` function,
- The `mat` is like this:

Y	X	Z	T
U	W	V	///

- Random number is 2, so empty cell should be moved right
- It's impossible, it's already in the right-most column, so ignore the move/swap, just return the same `mat`.
- Better luck in the next iteration. Finish.

Another function that you might need is a function to find '0' in your puzzle/matrix. The output would be a list or tuple which contains [row,col] of '0' in your 7-puzzle. We know that there is one and only one '0' in the puzzle.

```
def where_is(mat , 0)  
  
    return ii,jj
```

Test Cases

- You can test it with any 2x8 matrix of 0 to 7, but be careful that there could be no way to sort some of them! Why? Just think about that.
- These test cases however, are sortable for sure:

```
Mat1 = [ [1,2,3,4] , [5,6,7,0] ]  
Mat2 = [ [1,2,3,4] , [5,6,0,7] ]  
Mat3 = [ [1,6,2,4] , [5,3,0,7] ]  
Mat4 = [ [5,1,6,4] , [0,3,2,7] ]
```

- Try all of those patterns. The result might be different from what you expected regarding the number of moves needed to sort a simple puzzle. That's why AI is a bit complicated.
- Run your program a few times for each test case puzzle, apart from the `Mat1` (Why? ☺)
- You need to **run** the module every time. Otherwise, you may start from the last `Mati` (i=1 to 4, the test cases) generated by your last run instead of starting from the defined `Mati`
- You may need to go for a big `loop_end_limit` variable.

- Add some comments at the end of your program and report the average iterations that your program needs to sort any **Mati** test case. Of course, if it's been possible to sort that **Mati** in a reasonable time/iterations.
- **Basically**, we expect your program to be able to handle **Mat1** and **Mat2**. About two others it depends on your code and your chance.

Question 5: Index or Key Sorting [1 CA Mark]

Consider that we have got a big matrix/2d list of data like this that should be sorted:
The number of rows could easily be a few thousand.

company	body-style	wheel-base	length	engine-type	num-of-cylinders	horsepower	average-mileage	price
chevrolet	hatchback	88.4	141.1	l	three	48	47	5151
mazda	hatchback	93.1	159.1	ohc	four	68	30	5195
toyota	hatchback	95.7	158.7	ohc	four	62	35	6348
mitsubishi	hatchback	93.7	157.3	ohc	four	68	37	5389
mazda	hatchback	93.1	159.1	ohc	four	68	31	6095
mitsubishi	hatchback	93.7	157.3	ohc	four	68	31	5889
dodge	hatchback	93.7	157.3	ohc	four	68	31	6229
chevrolet	hatchback	94.5	155.9	ohc	four	70	38	6295
toyota	hatchback	95.7	158.7	ohc	four	62	31	6338
dodge	hatchback	93.7	157.3	ohc	four	68	31	7377
toyota	hatchback	95.7	158.7	ohc	four	62	31	6488
chevrolet	sedan	94.5	158.8	ohc	four	70	38	6575

Typical sort algorithms have got 2 problems to handle such a case:

- 1- Many sorting keys, for instance, back to the table above, if the 'company' is the same for a few records, the algorithm should refer to 'body_style' and then the next columns to decide.
- 2- Displacement of big records of data to sort the table is time-consuming.

You are going to develop a sort algorithm to address both concerns:

- We use a selection or merge sort as the base of our algorithm. (other sort algorithms can be used too)
- To compare 2 records, your code should compare the 1st column firstly, if they are equal the 2nd column should be considered, again in the case of equality it should refer to the next column. Although in many cases just the 1st column is enough to decide.
- To make the algorithm faster, we add an index or key array. The length of that is equal to the number of rows/records of the main table (n), and would be initiated with the record number or 0 to n-1:

Index	company	body-style	wheel-base	length	engine-type	num-of- cylinders	horsepower	average- mileage	price
0	chevrolet	hatchback	88.4	141.1	l	three	48	47	5151
1	mazda	hatchback	93.1	159.1	ohc	four	68	30	5195
2	toyota	hatchback	95.7	158.7	ohc	four	62	35	6348
3	mitsubishi	hatchback	93.7	157.3	ohc	four	68	37	5389
4	mazda	hatchback	93.1	159.1	ohc	four	68	31	6095
5	mitsubishi	hatchback	93.7	157.3	ohc	four	68	31	5889
6	dodge	hatchback	93.7	157.3	ohc	four	68	31	6229
7	chevrolet	hatchback	94.5	155.9	ohc	four	70	38	6295
8	toyota	hatchback	95.7	158.7	ohc	four	62	31	6338
9	dodge	hatchback	93.7	157.3	ohc	four	68	31	7377
10	toyota	hatchback	95.7	158.7	ohc	four	62	31	6488
11	chevrolet	sedan	94.5	158.8	ohc	four	70	38	6575

- Now, we can compare the records, but instead of swapping/displacing them, we just swap/displace their indexes in the index list/array.
- From the beginning, all the references to the main table records should be done via their indexes (not directly).

Definition:

```
def index_sort( TM ): pass
    return ind
```

Usage:

```
ind_key = index_sort( TM )
```

- You are not allowed to change the input matrix. i.e., after the sorting, only the index array is sorted (based on the contents of that table).
- How we can use it? Have a look:

```
ind_key= index_sort(
    table_matrix ) for i in range(
    len(ind_key) ):
    print( table_matrix[ind_key[i]] )
```

Example 1:

```
def index_sort(
    table_matrix ):
    pass
    return ind

-----
#
table2 = [ ['Miley Cyrus',164,106] , ['Geri Halliwell', 157.5,107] ,
['Madonna',162.6,107], ['Fergie/Stacy Ferguson',158.8,108] , ['Victoria
Beckham',167.6,108] , ['Penelope Cruz',172.7,109] , ['ELVIS PRESLEY',182.2,165] ,
['CHARLIE CHAPLIN',162.6,165],
['LARRY FINE', 163,165] , ['KIEFER SUTHERLAND',174,168] , ['Raven
Symone',180.3,170], ['KURT RUSSELL',176,170] , ['John Wayne',193,170] ]
```

```

for i in range( len( aa ) ):
    print( table2[i] )
print("\n=====\\n')
for i in range( len( aa ) ):
    print( table2[aa[i]] )

#----- Results: -----
['Miley Cyrus', 164, 106]
['Geri Halliwell', 157.5, 107]
['Madonna', 162.6, 107]
['Fergie/Stacy Ferguson', 158.8, 108]
['Victoria Beckham', 167.6, 108]
['Penelope Cruz', 172.7, 109]
['ELVIS PRESLEY', 182.2, 165]
['CHARLIE CHAPLIN', 162.6, 165]
['LARRY FINE', 163, 165]
['KIEFER SUTHERLAND', 174, 168]
['Raven Symone', 180.3, 170]
['KURT RUSSELL', 176, 170]
['John Wayne', 193, 170]

=====

['CHARLIE CHAPLIN', 162.6, 165]
['ELVIS PRESLEY', 182.2, 165]
['Fergie/Stacy Ferguson', 158.8, 108]
['Geri Halliwell', 157.5, 107]
['John Wayne', 193, 170]
['KIEFER SUTHERLAND', 174, 168]
['KURT RUSSELL', 176, 170]
['LARRY FINE', 163, 165]
['Madonna', 162.6, 107]
['Miley Cyrus', 164, 106]
['Penelope Cruz', 172.7, 109]
['Raven Symone', 180.3, 170]
['Victoria Beckham', 167.6, 108]

```

Example 2:

```

def index_sort( table_matrix ):
    pass
    return ind

#-----

table1 = [ ['chevrolet','hatchback',88.4,141.1,'l','three',48,47,5151],
['mazda', 'hatchback', 93.1, 159.1,'ohc','four',68,30,5195],
['toyota','hatchback',95.7,158.7,'ohc','four', 62,35,6348],
['mitsubishi', 'hatchback',93.7,157.3,'ohc','four',68,37,5389],
['mazda','hatchback',93.1,159.1,'ohc','four',68,31,6095] ,
['mitsubishi','hatchback',93.7,157.3,'ohc','four',68,31,5889] ,
['dodge','hatchback',93.7,157.3,'ohc','four',68,31,6229] ,
['chevrolet','hatchback',94.5,155.9,'ohc','four',70,38,6295] ,
['toyota','hatchback',95.7,158.7,'ohc','four',62,31,6338] ,
['dodge','hatchback',93.7,157.3,'ohc','four',68,31,7377] ,
['toyota','hatchback',95.7,158.7,'ohc','four',62,31,6488] ,
['chevrolet','sedan',94.5,158.8,'ohc','four',70,38,6575] ]

```

```
#-----
aa= index_sort( table1 )

for i in range( len( aa ) ):
    print( table1[i] )
print('\n=====')

for i in range( len( aa ) ):
    print( table1[aa[i]] )

===== Results =====
['chevrolet', 'hatchback', 88.4, 141.1, 'l', 'three', 48, 47, 5151]
['mazda', 'hatchback', 93.1, 159.1, 'ohc', 'four', 68, 30, 5195]
['toyota', 'hatchback', 95.7, 158.7, 'ohc', 'four', 62, 35, 6348]
['mitsubishi', 'hatchback', 93.7, 157.3, 'ohc', 'four', 68, 37, 5389]
['mazda', 'hatchback', 93.1, 159.1, 'ohc', 'four', 68, 31, 6095]
['mitsubishi', 'hatchback', 93.7, 157.3, 'ohc', 'four', 68, 31, 5889]
['dodge', 'hatchback', 93.7, 157.3, 'ohc', 'four', 68, 31, 6229]
['chevrolet', 'hatchback', 94.5, 155.9, 'ohc', 'four', 70, 38, 6295]
['toyota', 'hatchback', 95.7, 158.7, 'ohc', 'four', 62, 31, 6338]
['dodge', 'hatchback', 93.7, 157.3, 'ohc', 'four', 68, 31, 7377]
['toyota', 'hatchback', 95.7, 158.7, 'ohc', 'four', 62, 31, 6488]
['chevrolet', 'sedan', 94.5, 158.8, 'ohc', 'four', 70, 38, 6575]

=====
['chevrolet', 'hatchback', 88.4, 141.1, 'l', 'three', 48, 47, 5151]
['chevrolet', 'hatchback', 94.5, 155.9, 'ohc', 'four', 70, 38, 6295]
['chevrolet', 'sedan', 94.5, 158.8, 'ohc', 'four', 70, 38, 6575]
['dodge', 'hatchback', 93.7, 157.3, 'ohc', 'four', 68, 31, 6229]
['dodge', 'hatchback', 93.7, 157.3, 'ohc', 'four', 68, 31, 7377]
['mazda', 'hatchback', 93.1, 159.1, 'ohc', 'four', 68, 30, 5195]
['mazda', 'hatchback', 93.1, 159.1, 'ohc', 'four', 68, 31, 6095]
['mitsubishi', 'hatchback', 93.7, 157.3, 'ohc', 'four', 68, 31, 5889]
['mitsubishi', 'hatchback', 93.7, 157.3, 'ohc', 'four', 68, 37, 5389]
['toyota', 'hatchback', 95.7, 158.7, 'ohc', 'four', 62, 31, 6338]
['toyota', 'hatchback', 95.7, 158.7, 'ohc', 'four', 62, 31, 6488]
['toyota', 'hatchback', 95.7, 158.7, 'ohc', 'four', 62, 35, 6348]
```

Points:

- Pay attention to how the index list has been used to access the records/rows of the table.
- Pay attention to how more columns get involved in sorting when the first ones are equal.

Results:

Your function just **should return** a sorted index list and this would be tested to see if it works properly or not.
For instance:

• Example 1 output: [7, 6, 3, 1, 12, 9, 11, 8, 2, 0, 5, 10, 4]	• Example 2 output: [0, 7, 11, 9, 6, 1, 4, 3, 5, 2, 10, 8]
--	--

GOOD LUCK MY FRIENDS